

Pre-entrega TPE · Protección de Servicios con WAF para The Store

Tema 9 · Redes de Información · ITBA · 1C 2026
Grupo 2

Integrantes: [Completar integrantes]

1. Problemática y contexto

- The Store corre sobre Kind/Kubernetes y expone solo la UI vía `ingress-nginx`; los demás microservicios quedan como `ClusterIP`.
- La auditoría pre-WAF ya realizada sobre `http://localhost` confirmó 10 vectores explotables sin autenticación. Los más críticos son: SSRF/path traversal vía `/proxy/**`, IDOR de lectura sobre `/proxy/carts/{customerId}`, y exposición de Spring Actuator (`info`, `health`, `metrics`, `prometheus`).
- Hoy tampoco hay rate limiting, detección de scanners conocidos ni headers de seguridad HTTP. Esto deja a la aplicación expuesta a enumeración, scraping, abuso automatizado y fuga de información.
- El problema es transversal a una arquitectura multi-lenguaje (Java, Go y Node.js): corregir en cada servicio implica tocar varias bases de código, mientras que un WAF permite aplicar una política única en el punto de entrada.

2. Diseño de la solución

- Se propone desplegar `ModSecurity v3` con `OWASP Core Rule Set (CRS)` como módulo del `ingress-nginx-controller` ya presente en el cluster.
- La solución operará en dos etapas: `DetectionOnly` para tuning inicial y luego `On` para bloqueo efectivo. El objetivo es minimizar falsos positivos antes de la demo final.
- Además del CRS base, se sumarán reglas custom de bajo acoplamiento al negocio para: bloquear `/actuator/*` sensibles, negar traversal en `/proxy/**`, limitar abuso automatizado y reforzar headers de seguridad.
- La ventaja principal es que no requiere cambios funcionales en los microservicios: el cambio queda concentrado en `ingress-nginx` (`ConfigMap`, anotaciones del Ingress y reglas `ModSecurity`).

3. Scope del POC y casos de uso

Alcance comprometido

- Bloqueo de Actuator sensible:** Requests a `/actuator/info`, `/actuator/metrics` y `/actuator/prometheus` deberán pasar de `200 OK` a `403 Forbidden`.
- Mitigación de SSRF/path traversal vía proxy:** Payloads como `/proxy/catalog/../../actuator/prometheus` deberán quedar bloqueados por CRS 930xxx y/o regla custom específica.
- Detección de scanners conocidos:** User-Agents de herramientas como Nikto o sqlmap deberán quedar registrados y bloqueados por reglas del CRS.
- Rate limiting en el punto de entrada:** Se limitará el caudal por IP desde el Ingress para mitigar abuso automatizado y DoS de baja complejidad.
- Headers de seguridad HTTP:** El Ingress agregará `HSTS`, `X-Frame-Options`, `X-Content-Type-Options`, `Content-Security-Policy`, `Referrer-Policy` y `Permissions-Policy`.

Casos de uso para la demo

- Antes/después de exfiltración de métricas vía `/actuator/prometheus`.
- Antes/después de traversal vía `/proxy/catalog/../../actuator/info`.
- Request con User-Agent de Nikto antes/después del WAF.
- Burst de requests para mostrar ausencia/presencia de rate limiting.
- Comparación de headers de respuesta de la home antes/después.

4. Diagrama de arquitectura



Bloque	CIDR	Rol
Host loopback	127.0.0.1/32	Usuario accede a The Store por 80/443

Bloque	CIDR	Rol
Docker / nodo Kind	172.18.0.0/16	Red del contenedor del nodo
Pods	10.244.0.0/16	Comunicación entre pods
Services	10.96.0.0/12	ClusterIP internos

- **SO base del nodo:** `kindest/node` sobre Ubuntu 22.04.
- **Protocolos:** HTTP/1.1 externo e interno; DNS interno de Kubernetes.
- **Punto de inserción del WAF:** `ingress-nginx-controller` en el namespace `ingress-nginx`.
- **Cambios esperados:** ConfigMap de `ingress-nginx`, reglas ModSecurity/CRS y anotaciones del Ingress principal.

5. Alternativas consideradas

- **Shadow Daemon:** Se descarta porque exige instrumentación más cercana a la aplicación y no encaja tan naturalmente con una arquitectura Java/Go/Node detrás de un Ingress compartido.
- **OpenWAF:** Fue considerado por figurar en el enunciado, pero se evita por menor madurez de ecosistema y menor predictibilidad para una demo académica.
- **Coraza:** Es técnicamente interesante y compatible con CRS, pero no figura en el tema asignado. Se deja como evolución futura, no como base del TPE.

6. Plan breve de validación

- Usar como baseline la evidencia pre-WAF ya capturada en `pre-analysis/evidencias/`.
- Repetir los PoCs seleccionados con `DetectionOnly` y luego con bloqueo activo.
- Guardar snapshot post-WAF y comparar resultados pre/post con un diff simple.
- Adjuntar logs de ModSecurity para demostrar que el bloqueo provino del WAF.

7. Criterio de éxito

Se considerará cumplido el POC si los vectores hoy explotables en el punto de entrada HTTP quedan detectados o bloqueados por el WAF, sin romper el acceso normal a la home, catálogo, carrito y checkout. La entrega final incluirá demo funcional, código en GitHub y how-to de despliegue.